

■ GUIDE DE DEVELOPPEMENT

Pour Assistant IA (Kimi / ChatGPT / Claude)
Projet IaC Chatbot - 100% Gratuit

Ce document est conçu pour être donné à un assistant IA (Kimi, ChatGPT, Claude, etc.) afin qu'il comprenne le projet en entier et puisse guider le développement étape par étape. Il contient toutes les spécifications techniques, les choix architecturaux, et les instructions précises pour générer du code fonctionnel.

Projet	Infrastructure as Code & Automation pilotée par Chatbot Intelligent
Stack	Spring Boot 3.2 + Angular 17 + Ollama (Llama 3) + MySQL Community + Jenkins Open Source
Cout	0€ - Tous les outils sont open-source ou gratuits
Duree	1 mois (20 jours ouvrés)
Objectif IA	Guider le développement, générer du code, expliquer les concepts, debugger

1. Contexte et Objectifs pour l'IA

1.1 Role de l'IA dans ce projet

L'assistant IA (Kimi) doit agir comme un **mentor technique senior** et un **generateur de code**. Il doit comprendre le projet dans sa globalite et etre capable de :

- **Generer du code** : Classes Java, composants Angular, scripts Terraform/YAML, configurations
- **Expliquer les concepts** : Spring Boot, Angular, Ollama, Jenkins, Docker, etc.
- **Debugger** : Analyser les erreurs et proposer des solutions
- **Guider l'architecture** : Conseiller sur les patterns, les bonnes pratiques
- **Revoir le code** : Suggestions d'amelioration, detection de bugs
- **Documenter** : Generer des commentaires, de la documentation technique

1.2 Connaissances requises par l'IA

L'IA doit maitriser les domaines suivants :

Domaine	Niveau requis	Details
Java / Spring Boot	Expert	Spring Boot 3.2, JPA, Security, WebSocket, REST API, Lombok
TypeScript / Angular	Expert	Angular 17+, RxJS, HTTP Client, WebSocket, Components
LLM / IA	Avance	Ollama, Llama 3, Spring AI, prompt engineering
DevOps	Intermediaire	Docker, Jenkins, Terraform, Kubernetes/Minikube
Bases de donnees	Intermediaire	MySQL, H2, JPA/Hibernate, Flyway
Securite	Intermediaire	JWT, Spring Security, BCrypt, CORS

1.3 Format des requetes a l'IA

Pour obtenir les meilleures reponses de l'IA, utiliser ce format :

- **Generer du code** : "Genere la classe User avec JPA, Lombok, validation des champs, et les methodes authenticate() et hasReachedQuota()"
- **Expliquer** : "Explique comment fonctionne Spring AI avec Ollama, donne un exemple de ChatClient"
- **Debugger** : "J'ai cette erreur [STACKTRACE], analyse et propose une solution"
- **Architecture** : "Quel pattern utiliser pour le service de deploiement : Strategy ou Template Method ?"
- **Revue** : "Revois ce code [CODE] et suggere des ameliorations de performance et de securite"

2. Architecture Detaillee pour l'IA

2.1 Structure du projet Spring Boot

L'IA doit generer ce package structure exact :

```
iac-chatbot-backend/  
■■■ src/main/java/com/company/iacchatbot/  
■ ■■■ IacChatbotApplication.java  
■ ■■■ config/  
■ ■ ■■■ SecurityConfig.java # JWT + CORS + Role-based access  
■ ■ ■■■ WebSocketConfig.java # STOMP endpoints  
■ ■ ■■■ OllamaConfig.java # Spring AI Ollama configuration  
■ ■■■ controller/  
■ ■ ■■■ AuthController.java # POST /api/auth/login, /register  
■ ■ ■■■ ChatController.java # WebSocket /app/chat + @MessageMapping  
■ ■ ■■■ DeployController.java # POST /api/deploy, GET /api/deploy/{id}/status  
■ ■ ■■■ HistoryController.java # GET /api/history, GET /api/history/{userId}  
■ ■ ■■■ AdminController.java # GET /api/admin/pending, POST /api/admin/approve/{id}  
■ ■■■ dto/  
■ ■ ■■■ ChatRequest.java # {message: String, language: String}  
■ ■ ■■■ ChatResponse.java # {response: String, params: Map, code: String, status: String}  
■ ■ ■■■ DeployRequest.java # {deploymentId: String, code: String, platform: String}  
■ ■ ■■■ DeployResponse.java # {deploymentId: String, status: String, message: String}  
■ ■■■ model/  
■ ■ ■■■ User.java # JPA Entity with quotas  
■ ■ ■■■ DeploymentRequest.java # JPA Entity with all params  
■ ■ ■■■ IaCTemplate.java # JPA Entity for templates  
■ ■ ■■■ DeploymentLog.java # JPA Entity for logs  
■ ■ ■■■ enums/  
■ ■ ■■■ ResourceType.java # VM, CONTAINER  
■ ■ ■■■ PlatformType.java # VSPHERE, OPENSIFT  
■ ■ ■■■ DeploymentStatus.java # PENDING, CODE_GENERATED, RUNNING, SUCCESS, FAILED, CANCELLED  
■ ■ ■■■ Role.java # USER, ADMIN  
■ ■ ■■■ LogLevel.java # INFO, WARN, ERROR  
■ ■■■ repository/  
■ ■ ■■■ UserRepository.java # extends JpaRepository  
■ ■ ■■■ DeploymentRequestRepository.java  
■ ■ ■■■ IaCTemplateRepository.java  
■ ■ ■■■ DeploymentLogRepository.java  
■ ■■■ service/  
■ ■ ■■■ AuthService.java # login, register, JWT generation  
■ ■ ■■■ ChatService.java # orchestrate chat flow  
■ ■ ■■■ LlmService.java # Ollama ChatClient calls  
■ ■ ■■■ IaCGeneratorService.java # Thymeleaf template rendering  
■ ■ ■■■ DeployService.java # ProcessBuilder for terraform/oc  
■ ■ ■■■ HistoryService.java # query deployment history  
■ ■ ■■■ NotificationService.java # Discord Webhook + MailHog  
■ ■■■ security/  
■ ■ ■■■ JwtTokenProvider.java # generate/validate JWT  
■ ■ ■■■ JwtAuthenticationFilter.java # filter chain  
■ ■■■ util/  
■ ■■■ ProcessExecutor.java # execute shell commands safely  
■■■ src/main/resources/  
■ ■■■ templates/  
■ ■ ■■■ terraform/  
■ ■ ■ ■■■ vmware-vm.tf # Thymeleaf template  
■ ■ ■■■ openshift/  
■ ■ ■■■ deployment.yaml # Thymeleaf template  
■ ■ ■■■ service.yaml
```

- ■ ■■ route.yaml
- ■ ■■ kubevirt-vm.yaml
- ■■ application.properties
- ■■ application-dev.properties # H2 config
- ■■ application-prod.properties # MySQL config
- src/test/java/... # Unit + Integration tests
- Dockerfile
- docker-compose.yml # Jenkins + MailHog + App
- Jenkinsfile
- pom.xml

2.2 Structure du projet Angular

```

iac-chatbot-frontend/
■■■ src/app/
■ ■■ app.component.ts/html/css
■ ■■ app-routing.module.ts
■ ■■ app.module.ts
■ ■■ core/ # Singleton services
■ ■ ■■ services/
■ ■ ■ ■■ auth.service.ts # JWT login/logout
■ ■ ■ ■■ chat.service.ts # WebSocket STOMP
■ ■ ■ ■■ deploy.service.ts # HTTP deploy calls
■ ■ ■ ■■ history.service.ts # HTTP history calls
■ ■ ■■ guards/
■ ■ ■ ■■ auth.guard.ts # Route protection
■ ■ ■■ interceptors/
■ ■ ■■ jwt.interceptor.ts # Add JWT to headers
■ ■■ features/
■ ■ ■■ chat/
■ ■ ■ ■■ chat.component.ts # Main chatbot UI
■ ■ ■ ■■ chat-message.component.ts
■ ■ ■ ■■ code-display.component.ts
■ ■ ■■ dashboard/
■ ■ ■ ■■ dashboard.component.ts
■ ■ ■ ■■ stats-cards.component.ts
■ ■ ■ ■■ deployments-table.component.ts
■ ■ ■■ admin/
■ ■ ■ ■■ admin-panel.component.ts
■ ■ ■ ■■ user-management.component.ts
■ ■ ■ ■■ approval-queue.component.ts
■ ■ ■■ auth/
■ ■ ■■ login.component.ts
■ ■ ■■ register.component.ts
■ ■■ shared/
■ ■ ■■ models/
■ ■ ■ ■■ chat-request.model.ts
■ ■ ■ ■■ chat-response.model.ts
■ ■ ■ ■■ deployment.model.ts
■ ■ ■■ components/
■ ■ ■■ header.component.ts
■ ■ ■■ sidebar.component.ts
■ ■■ assets/
■ ■■ i18n/ # fr.json, en.json
■■■ angular.json
■■■ package.json
■■■ Dockerfile

```

3. Instructions de Generation de Code

3.1 Entites JPA (a generer par l'IA)

L'IA doit generer ces 4 entites avec Lombok, validation, et relations :

User.java - Specifications exactes :

- @Entity + @Table(name = "users")
- @Id @GeneratedValue(strategy = IDENTITY) private Long id
- @Column(unique=true, nullable=false, length=50) private String username
- @Column(unique=true, nullable=false) private String email
- @Column(nullable=false) private String password (BCrypt hash)
- @Enumerated(STRING) @Column(nullable=false) private Role role (USER/ADMIN)
- private Integer quotaCpu = 32, quotaRam = 128, quotaStorage = 1000
- private Boolean active = true
- @OneToMany(mappedBy="user", cascade=ALL) private List deployments
- Methodes : authenticate(), hasReachedQuota(), getRemainingQuota()

DeploymentRequest.java - Specifications exactes :

- @Entity + @Table(name = "deployment_requests")
- @Id @GeneratedValue(strategy = IDENTITY) private Long id
- @ManyToOne @JoinColumn(name="user_id") private User user
- @Column(nullable=false, length=500) private String requestText
- @Enumerated(STRING) private ResourceType resourceType
- @Enumerated(STRING) private PlatformType platform
- @Min(1) @Max(32) private Integer cpu = 2
- @Min(1) @Max(128) private Integer ramGb = 4
- @Min(10) @Max(1000) private Integer storageGb = 50
- private String osImage = "ubuntu-22.04", network = "default"
- @Min(1) @Max(10) private Integer replicas = 1
- @Column(columnDefinition = "TEXT") private String generatedCode
- @Enumerated(STRING) private DeploymentStatus status = PENDING
- private String deploymentId (UUID)
- @OneToMany(mappedBy="deploymentRequest", cascade=ALL) private List logs

3.2 Service LLM (Ollama) - a generer par l'IA

L'IA doit generer ce service avec Spring AI :

```
@Service
@RequiredArgsConstructor
public class LlmService {

    private final ChatClient chatClient;

    public InfrastructureParams extractParams(String userMessage) {
        String prompt = buildExtractionPrompt(userMessage);
        String response = chatClient.call(prompt);
        return parseJsonResponse(response);
    }

    private String buildExtractionPrompt(String message) {
        return """"Tu es un assistant expert en infrastructure cloud.
Analyse cette demande et extrais les parametres JSON:

Demande: """ + message + """"
```

Reponds UNIQUEMENT avec ce JSON:

```
{
  "resourceType": "vm" ou "container",
  "platform": "vsphere" ou "openshift",
  "cpu": nombre (default: 2),
  "ramGb": nombre (default: 4),
  "storageGb": nombre (default: 50),
  "osImage": "ubuntu-22.04" ou autre,
  "network": "default",
  "replicas": nombre (default: 1)
}""";
}
```

3.3 Template Terraform (Thymeleaf) - a generer par l'IA

```
terraform {
  required_providers {
    vsphere = { source = "hashicorp/vsphere", version = "~> 2.0" }
  }
}

provider "vsphere" {
  user = var.vsphere_user
  password = var.vsphere_password
  vsphere_server = var.vsphere_server
  allow_unverified_ssl = true
}

data "vsphere_datacenter" "dc" { name = var.datacenter_name }
data "vsphere_datastore" "ds" {
  name = var.datastore_name
  datacenter_id = data.vsphere_datacenter.dc.id
}
data "vsphere_network" "net" {
  name = var.network_name
  datacenter_id = data.vsphere_datacenter.dc.id
}
data "vsphere_virtual_machine" "template" {
  name = "[[${osImage}]]"
  datacenter_id = data.vsphere_datacenter.dc.id
}

resource "vsphere_virtual_machine" "vm" {
  name = "vm-[[${osImage}]]-[[${#dates.format(#dates.createNow(), 'yyyyMMdd-HHmss')}]]"
  resource_pool_id = var.resource_pool_id
  datastore_id = data.vsphere_datastore.ds.id
  num_cpus = [[${cpu}]]
  memory = [[${ramGb}]] * 1024
  guest_id = data.vsphere_virtual_machine.template.guest_id

  network_interface {
    network_id = data.vsphere_network.net.id
  }

  disk {
    label = "disk0"
  }
}
```

```
size = [[${storageGb}]]
thin_provisioned = true
}

clone {
  template_uuid = data.vsphere_virtual_machine.template.id
  customize {
    linux_options {
      host_name = "vm-[[${osImage}]]"
      domain = "company.local"
    }
    network_interface {}
  }
}
}
```

4. Prompts Prêts a l'Emploi pour l'IA

Copier-coller ces prompts dans Kimi/ChatGPT pour obtenir du code immediat :

PROMPT 1 : Generer les entites JPA completes

Genere les 4 entites JPA pour un projet Spring Boot avec Lombok et validation:

1. User (id, username, email, password, role:Enum{USER,ADMIN}, quotaCpu, quotaRam, quotaStorage, active, List)
2. DeploymentRequest (id, user, requestText, resourceType:Enum{VM,CONTAINER}, platform:Enum{VSPHERE,OPENSIFT}, cpu, ramGb, storageGb, osImage, network, replicas, generatedCode, status:Enum, deploymentId, completedAt, errorMessage, List)
3. IaCTemplate (id, name, resourceType, platform, templateContent, version, isActive)
4. DeploymentLog (id, deploymentRequest, step, logLevel:Enum{INFO,WARN,ERROR}, message, timestamp)

Inclut : annotations JPA, relations, contraintes de validation (@Min, @Max, @NotNull), et Lombok (@Data, @Builder, @NoArgsConstructor, @AllArgsConstructor).

PROMPT 2 : Generer le service LLM Ollama

Genere un service Spring Boot qui utilise Spring AI avec Ollama (modele llama3) pour extraire des parametres d'infrastructure d'une demande en langage naturel.

Le service doit:

- Utiliser ChatClient de Spring AI
- Construire un prompt qui demande un JSON structure
- Parser la reponse JSON en objet Java (InfrastructureParams)
- Gerer les erreurs (fallback sur valeurs par default)
- Etre annoté @Service avec Lombok @RequiredArgsConstructor

Configuration: spring.ai.ollama.base-url=http://localhost:11434, model=llama3

PROMPT 3 : Generer le controller REST complet

Genere les 5 controllers REST pour un projet Spring Boot avec Spring Security JWT:

1. AuthController : POST /api/auth/login, POST /api/auth/register
2. ChatController : WebSocket @MessageMapping("/chat") pour chatbot temps reel
3. DeployController : POST /api/deploy, GET /api/deploy/{id}/status, DELETE /api/deploy/{id}
4. HistoryController : GET /api/history, GET /api/history/user/{userId}
5. AdminController : GET /api/admin/pending, POST /api/admin/approve/{id}, POST /api/admin/reject/{id}

Inclut : validation des DTO, gestion des exceptions, annotations @PreAuthorize pour les roles.

PROMPT 4 : Generer le composant Angular Chatbot

Genere un composant Angular 17 pour un chatbot interactif avec:

- Zone de messages (style ChatGPT, bulles utilisateur/bot)
- Champ de saisie avec envoi par Enter
- Affichage du code genere (Terraform/YAML) avec coloration
- Bouton "Deployer" qui declenche l'appel API
- Indicateur de chargement pendant le traitement
- WebSocket STOMP pour communication temps reel
- Utilisation de RxJS et Angular Material

PROMPT 5 : Generer la configuration Securite JWT

Genere la configuration Spring Security avec JWT:

- SecurityConfig : filter chain, CORS, stateless session
- JwtTokenProvider : generer et valider les tokens

- JwtAuthenticationFilter : extraire et valider le token des requetes
- AuthService : login (verifie BCrypt), register (hash BCrypt)
- UserDetailsService implementation

Duree token : 24h. Algorithme : HS256. Secret depuis variable d'environnement.

PROMPT 6 : Generer le pipeline Jenkins

Genere un Jenkinsfile pour un projet Spring Boot + Angular:

Stages:

1. Checkout (Git)
2. Build Backend (Maven : mvn clean package)
3. Build Frontend (npm install, ng build)
4. Tests (mvn test, tests Angular)
5. Docker Build (docker build -t iac-chatbot)
6. Deploy (docker-compose up -d)

Inclut : notifications Discord webhook en cas de succes/echec.

5. Regles et Conventions pour l'IA

5.1 Conventions de code

- **Java** : Package com.company.iacchatbot, Lombok partout, Java 21, records pour les DTO
- **Angular** : Standalone components (pas de NgModule), signals pour l'etat, RxJS pour les streams
- **Nommage** : camelCase pour Java/TS, kebab-case pour fichiers, UPPER_SNAKE pour constantes
- **Tests** : JUnit 5 + Mockito (backend), Jasmine + Karma (frontend), couverture > 70%
- **Git** : Conventional Commits (feat:, fix:, docs:, test:, refactor:)
- **Documentation** : JavaDoc sur les classes publiques, README.md a jour, Swagger auto-generé

5.2 Patterns a utiliser

Pattern	Usage	Implementation
Repository	Acces donnees	Spring Data JpaRepository + custom queries
Service	Logique metier	@Service avec @RequiredArgsConstructor, transactions @Transactional
DTO	Transfert donnees	Records Java pour les request/response
Strategy	Generation IaC	Interface IaCGenerator + implementations Terraform/YAML
Observer	Notifications	Spring Events + @EventListener
Factory	Creation templates	IaCTemplateFactory selon resourceType + platform
Singleton	Services Spring	Par default avec @Service (scope singleton)

5.3 Gestion des erreurs

L'IA doit implementer cette strategie de gestion d'erreurs :

- **GlobalExceptionHandler** : @ControllerAdvice avec @ExceptionHandler pour chaque type d'erreur
- **Custom exceptions** : LlmExtractionException, DeploymentFailedException, QuotaExceededException
- **HTTP Status** : 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 500 (server error)
- **Response format** : {timestamp, status, error, message, path} pour toutes les erreurs
- **Logging** : SLF4J avec niveaux appropriés (INFO pour succes, WARN pour recoverable, ERROR pour critique)

5.4 Securite obligatoire

L'IA DOIT implementer ces mesures de securite :

- **JWT** : Token signe avec HS256, expiration 24h, refresh token optionnel
- **BCrypt** : Mots de passe hashes avec strength 12 (BCryptPasswordEncoder)
- **CORS** : Configuration restrictive (origines autorisees uniquement)
- **Validation** : @Valid sur tous les endpoints, @NotNull, @Size, @Pattern
- **SQL Injection** : JPA/Hibernate (pas de native queries sans parametres)
- **XSS** : Escaping automatique par Thymeleaf, validation des entrees
- **Secrets** : AUCUN secret en dur dans le code, tous dans variables d'environnement

6. Checklist de Validation du Code

Avant de considerer une tache comme terminee, l'IA doit verifier :

Categorie	Verification	Critere
Code Java	Compilation sans erreur	mvn clean compile reussit
Code Java	Tests unitaires passent	mvn test reussit (couverture > 70%)
Code Java	Pas de warnings critiques	SonarLint / IntelliJ inspections
Code Java	Lombok utilise correctement	@Data, @Builder, @RequiredArgsConstructor
Angular	Compilation sans erreur	ng build --configuration production reussit
Angular	Tests passent	ng test --watch=false --browsers=ChromeHeadless
Angular	Pas de memory leaks	Subscriptions unsubscribe dans ngOnDestroy
Securite	JWT fonctionnel	Login genere token valide, acces protege
Securite	Mots de passe chiffres	BCryptPasswordEncoder utilise
Securite	CORS configure	Origines restreintes, credentials autorises
API	Swagger documente	/swagger-ui.html accessible et complet
API	Endpoints RESTful	Noms coherents, methodes HTTP appropriees
BDD	Migrations Flyway	V1__init.sql present et executable
BDD	Contraintes definies	PK, FK, unique, not null, check
Docker	Image buildable	docker build -t iac-chatbot . reussit
Docker	Multi-stage build	Build Maven + runtime JRE separees
IaC	Templates valides	terraform validate + kubectl apply --dry-run
Documentation	README complet	Installation, demarrage, configuration
Git	Commits propres	Conventional commits, messages clairs

7. Exemple de Session de Developpement avec l'IA

Voici un exemple concret d'interaction avec Kimi pour developper le projet :

UTILISATEUR :

"Jour 6 du stage. Je dois creer le service LLM avec Ollama.
Voici ma configuration : spring.ai.ollama.base-url=http://localhost:11434, model=llama3.
Genere le service complet avec extraction JSON des parametres."

KIMI (IA) :

"Voici le service LlmService complet avec Spring AI et Ollama :

[Code genere avec :

- @Service + @RequiredArgsConstructor
- ChatClient injection
- Methode extractParams(String userMessage)
- Prompt engineering pour JSON structure
- Parsing JSON avec Jackson ObjectMapper
- Gestion d'erreurs avec fallback
- DTO InfrastructureParams]

UTILISATEUR :

"Merci ! Maintenant j'ai une erreur : 'Could not autowire. No beans of ChatClient type found.'"

KIMI (IA) :

"L'erreur vient de la configuration manquante. Ajoute cette classe @Configuration :

[Code genere avec :

- @Configuration
- @Bean ChatClient chatClient(OllamaChatClient ollamaChatClient)
- return ChatClient.builder(ollamaChatClient).build()]

UTILISATEUR :

"Ca marche ! Peux-tu maintenant generer le test unitaire pour ce service ?"

KIMI (IA) :

"Voici le test avec Mockito et JUnit 5 :

[Code genere avec :

- @ExtendWith(MockitoExtension.class)
- @Mock ChatClient
- @InjectMocks LlmService
- 3 tests : extraction reussie, JSON invalide, erreur reseau]

■ **Astuce :** Toujours fournir a l'IA :

- Le **contexte** (jour du stage, objectif du moment)
- La **configuration** (versions, proprietes, environnement)
- Le **code existant** (copier-coller des classes liees)
- L'**erreur exacte** (stacktrace complet si possible)
- Le **resultat attendu** (format de sortie, comportement)

8. Resume pour l'IA

Ce guide permet a un assistant IA de comprendre le projet IaC Chatbot dans son integralite et de generer du code de qualite professionnelle. L'IA doit agir comme un mentor technique qui guide le developpement etape par etape, en respectant les conventions et les patterns definis.

Element	Valeur
Projet	IaC Chatbot - Plateforme d'automatisation d'infrastructure pilotee par chatbot
Stack	Spring Boot 3.2 + Angular 17 + Ollama (Llama 3) + MySQL Community + Jenkins Open Source
Cout	0€ - Tous les outils sont open-source ou gratuits
Duree	1 mois (20 jours ouvrés)
Role de l'IA	Mentor technique, generateur de code, debugger, architecte conseil
Entites	User, DeploymentRequest, IaCTemplate, DeploymentLog (4 entites JPA)
Controllers	Auth, Chat (WebSocket), Deploy, History, Admin (5 controllers)
Services	Auth, Chat, Llm (Ollama), IaCGenerator, Deploy, History, Notification (7 services)
Templates IaC	Terraform (VMware), YAML Kubernetes, YAML KubeVirt (4 templates Thymeleaf)
Securite	JWT + Spring Security + BCrypt + CORS + Validation
CI/CD	Jenkins Open Source en Docker avec pipeline lint/test/build/deploy
Notifications	Discord Webhook (gratuit) + MailHog (dev)

Document prepare pour : Assistant IA (Kimi / ChatGPT / Claude)
Projet : Infrastructure as Code & Automation pilotee par Chatbot Intelligent
Version : 1.0 - Guide de developpement complet
Date : Aout 2026